

可以。下面我把它整理成一份可以直接作为项目 README / 开发设计文档的版本。目标不是做一个“玩具程序”，而是做一个规模很小、但物理和数值结构尽可能接近平面波 Kohn–Sham DFT/VASP 的 Mini-DFT。

# Mini-DFT 总体设计书

版本：v0.1  
定位：教学型 / 原型型平面波 Kohn–Sham DFT  
语言：Python  
核心库：NumPy + SciPy  
目标：通过亲手实现，理解 VASP 类平面波 DFT 从输入到 SCF 收敛的完整数值流程

## 1. 项目目标

### 1.1 核心目标

实现一个最小的三维周期性 Kohn–Sham DFT 程序，使其能够完成：

晶体结构 → 平面波基组 →  $H[n]$  → 本征值/波函数 →  $n(\mathbf{r})$  → SCF →  $E_{\text{total}}$

最终能够输出：

- Kohn-Sham 本征值
- Kohn-Sham 波函数
- 电荷密度
- Hartree 势
- XC 势
- 总能量
- Fermi energy（后期）
- DOS（后期）

### 1.2 非目标

第一版不是：

- VASP 的替代品
- 高性能 DFT 软件
- 真实材料数据库
- PAW 完整实现
- MPI/GPU 并程序序

我们故意牺牲：

计算规模

换取：

物理透明性

## 2. 第一版物理模型

采用：

3D periodic spin-unpolarized Kohn-Sham DFT

即：

- 三维周期体系
- 非自旋极化
- 单电子 Kohn-Sham 方程
- 平面波基组
- LDA
- 简单局域离子势
- SCF
- 数值本征值求解

Kohn-Sham 方程：

$$\left[ -\frac{\hbar^2}{2m} \nabla^2 + V_{\text{ion}}(\mathbf{r}) + V_H(\mathbf{r}) + V_{xc}(\mathbf{r}) \right] \psi_{n\mathbf{k}} = \epsilon_{n\mathbf{k}} \psi_{n\mathbf{k}}$$

其中：

$$V_{\text{eff}} = V_{\text{ion}} + V_H + V_{xc}.$$

### 3. 最重要的软件思想

整个程序采用：

Physics layer + Numerical layer

分离。

尤其是：

**DFT 程序不负责实现大型本征值算法。**

DFT 程序只负责告诉 eigensolver：

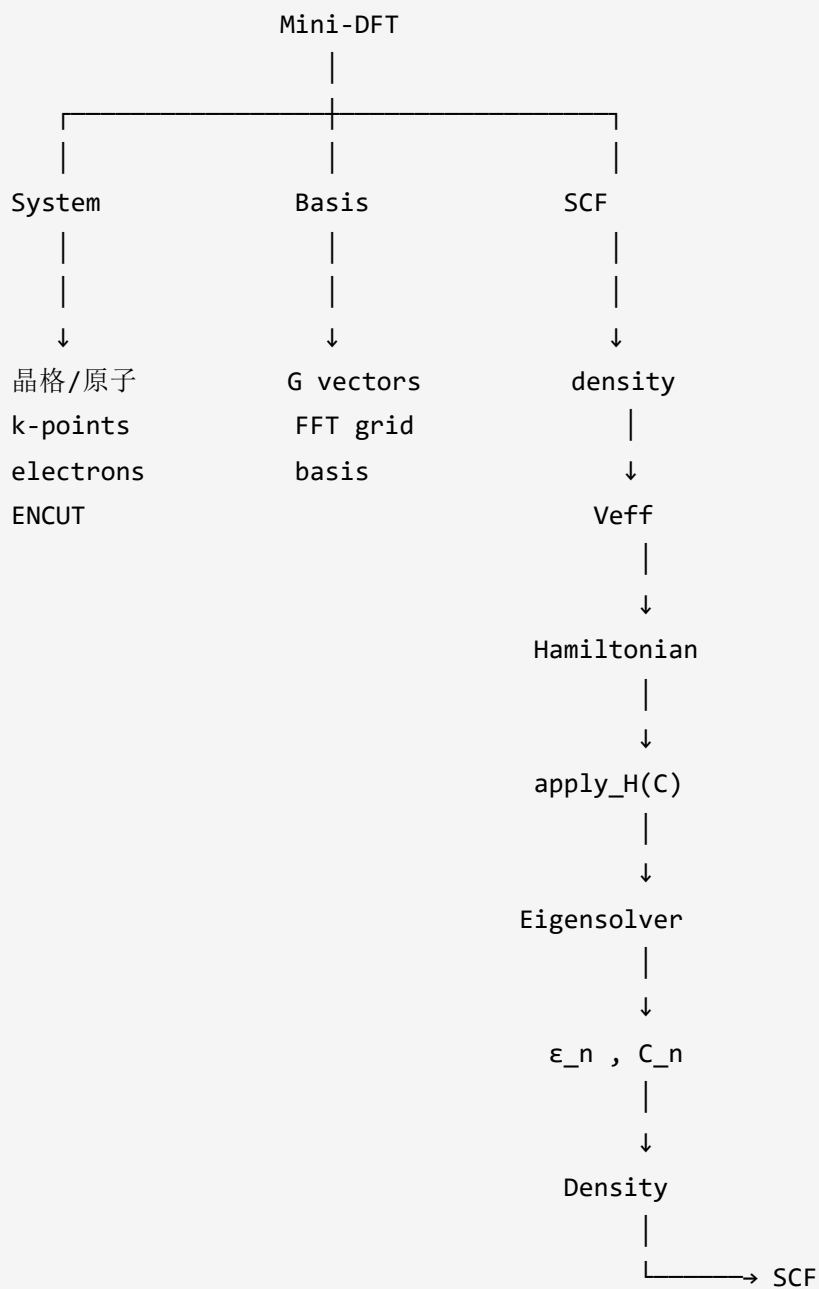
$$c \rightarrow Hc$$

然后 eigensolver 负责求：

$$HC = CE$$

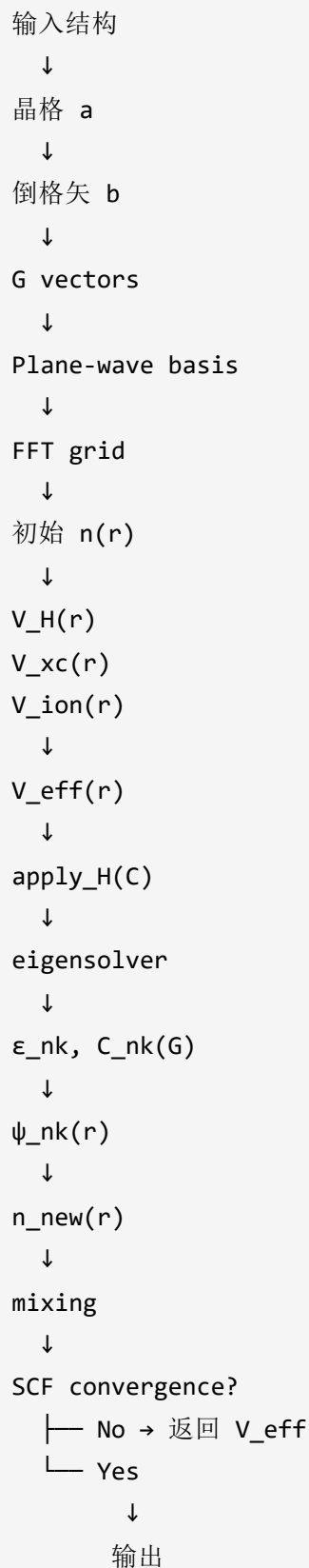
---

## 4. 总体架构



## 5. 数据流

整个程序最核心的数据流是：



---

## 6. 项目目录

推荐：

```
mini_dft/
|
├─ README.md
├─ pyproject.toml
|
├─ mini_dft/
|   └─ __init__.py
|       |
|       └─ system.py
|       └─ constants.py
|           |
|           └─ lattice.py
|           └─ basis.py
|           └─ fft_grid.py
|               |
|               └─ wavefunction.py
|               └─ density.py
|                   |
|                   └─ potentials.py
|                   └─ hartree.py
|                   └─ xc.py
|                   └─ ionic_potential.py
|                       |
|                       └─ hamiltonian.py
|                       └─ eigensolver.py
|                           |
|                           └─ occupations.py
|                           └─ mixing.py
|                           └─ energy.py
|                               |
|                               └─ scf.py
|                               └─ dos.py
|                                   |
|                                   └─ io.py
|
├─ examples/
|   └─ simple_cubic/
|   └─ one_atom/
|   └─ silicon/
|
└─ tests/
    └─ test_lattice.py
    └─ test_basis.py
    └─ test_fft.py
    └─ test_hamiltonian.py
    └─ test_hartree.py
```

```
└─ test_xc.py
└─ test_scf.py
```

第一版实际上只需要其中一部分。

## 7. System 模块

### 职责

描述物理体系。

核心数据：

$$\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3$$

原子位置：

$$\mathbf{R}_I$$

电子数：

$$N_e$$

k 点：

$$\mathbf{k}_i$$

k 点权重：

$$w_i$$

截断能：

$$E_{\text{cut}}$$

### 数据结构

概念上：

```
System(  
    lattice,  
    atoms,  
    electron_number,  
    kpoints,  
    kpoint_weights,  
    encut,  
)
```

---

## 8. Lattice 模块

给定：

$$A = [\mathbf{a}_1, \mathbf{a}_2, \mathbf{a}_3]$$

计算：

$$B = [\mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_3].$$

满足：

$$\mathbf{a}_i \cdot \mathbf{b}_j = 2\pi\delta_{ij}$$

因此：

$$B = 2\pi(A^{-1})^T.$$

这个模块还负责：

- 晶胞体积
- fractional  $\leftrightarrow$  Cartesian
- 周期坐标
- reciprocal lattice

---

## 9. Plane-wave Basis

这是整个程序最重要的基础模块之一。

波函数：



$$\psi_{n\mathbf{k}}(\mathbf{r}) = \sum_{\mathbf{G}} C_{n\mathbf{k}}(\mathbf{G}) e^{i(\mathbf{k}+\mathbf{G})\cdot\mathbf{r}}$$

无限多个 (G) 被截断。

## 9.1 ENCUT

用户指定：

$$E_{\text{cut}}.$$

保留：

$$\frac{\hbar^2}{2m} |\mathbf{k} + \mathbf{G}|^2 \leq E_{\text{cut}}$$

因此：

$$E_{\text{cut}} \rightarrow N_{\text{PW}}.$$

这意味着：

用户控制的是能量截断，而不是直接指定平面波数量。

## 10. FFT Grid

这是第二个非常重要的基础模块。

我们同时使用：

### Reciprocal space

$$C(\mathbf{G})$$

和：

### Real space

$$\psi(\mathbf{r}).$$

二者通过 FFT 转换：

$C(G) \leftrightarrow \psi(r)$

# 10.1 为什么需要 FFT?

因为：

## 动能项

在 reciprocal space 极其简单：

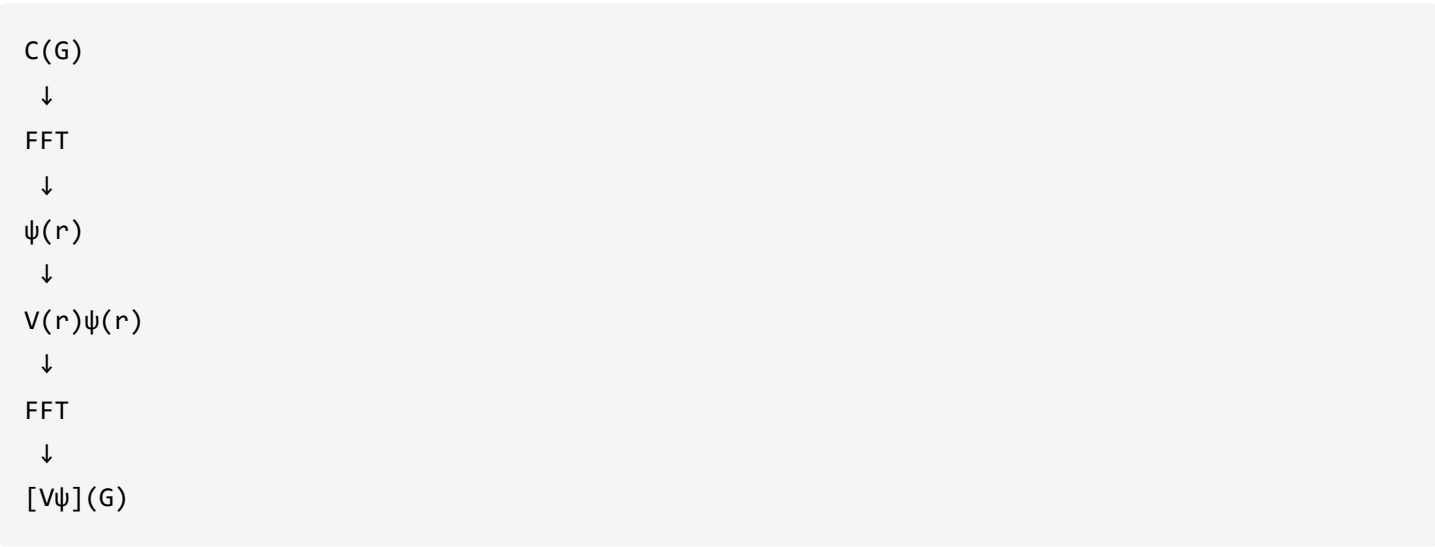
$$T_G = \frac{\hbar^2}{2m} |\mathbf{k} + \mathbf{G}|^2.$$

而局域势：

$$V(r)\psi(r)$$

在 real space 极其简单。

因此：



这正是平面波 DFT 的核心数值技巧。

# 11. Hamiltonian 模块

这是整个程序的核心。

我们不存：

$$H_{GG'}.$$

而实现：

$$H|\psi\rangle$$

即：

```
apply_H(C)
```

## 12. Hamiltonian 的计算

Kohn-Sham Hamiltonian：

$$H = T + V_{\text{ion}} + V_H + V_{xc}.$$

### 12.1 动能

$$(TC)_G = \frac{\hbar^2}{2m} |\mathbf{k} + \mathbf{G}|^2 C_G$$

直接逐元素乘法。

### 12.2 局域势

先：

$$C_G \rightarrow \psi(r).$$

然后：

$$V_{\text{eff}}(r)\psi(r).$$

最后：

$$\rightarrow [V_{\text{eff}}\psi]_G.$$

所以：

```
def apply_H(C):  
  
    kinetic = kinetic_operator(C)  
  
    psi_r = reciprocal_to_real(C)  
  
    potential_r = Veff_r * psi_r  
  
    potential_G = real_to_reciprocal(potential_r)  
  
    return kinetic + potential_G
```

---

## 13. 为什么不构造 H?

因为：

$$H \in \mathbb{C}^{N_{\text{PW}} \times N_{\text{PW}}}$$

如果：

$$N_{\text{PW}} = 100000,$$

那么显式矩阵巨大。

但我们实际上只需要：

$$HC.$$

因此采用：

matrix-free Hamiltonian

这是后面理解 VASP 的关键。

---

## 14. Eigensolver 模块

第一版：

SciPy eigsh

接口设计：

```
solve(  
    apply_H,  
    dimension,  
    n_bands,  
    tolerance,  
)
```

输出：

$$\epsilon_n$$

以及：

$$C_n.$$

---

## 14.1 与 DFT 解耦

程序绝不应该写成：

```
scipy.eigsh(...)
```

散落在 SCF 里面。

而应该：

```
SCF  
↓  
Eigensolver interface  
↓  
具体实现
```

未来可以替换：

SciPy eigsh

↓

Davidson

↓

LOBPCG

↓

其他库

而不修改 DFT 其他模块。

## 15. Density 模块

非自旋极化：

$$n(r) = 2 \sum_k w_k \sum_n f_{nk} |\psi_{nk}(r)|^2$$

第一版：

$$f_n = \begin{cases} 1 & \text{occupied} \\ 0 & \text{unoccupied} \end{cases}$$

所以：

$$n(r) = 2 \sum_k w_k \sum_{n \in \text{occupied}} |\psi_{nk}(r)|^2.$$

## 16. Occupation 模块

第一版：

绝缘体 + 非自旋极化 + 整数占据。

因此：

$$N_{\text{bands}}^{\text{occupied}} = N_e/2.$$

按照：

$$\epsilon_1 < \epsilon_2 < \dots$$

从低到高填充。

以后扩展：

- Fermi-Dirac
- Gaussian smearing
- Methfessel-Paxton
- tetrahedron

## 17. Hartree 模块

Hartree 势：

$$V_H(r) = e^2 \int \frac{n(r')}{|r - r'|} dr'.$$

使用 reciprocal space：

$$V_H(G) = \frac{4\pi e^2}{G^2} n(G)$$

因此：

```

n(r)
↓ FFT
n(G)
↓
4π/G²
↓
VH(G)
↓ inverse FFT
VH(r)

```

### 17.1 (G=0) 特殊处理

这里是第一个必须认真处理的数值问题：

$$\frac{4\pi}{G^2}$$

在：

$$G = 0$$

发散。

对于周期体系，通常需要考虑整体电中性以及离子背景。

第一版可以明确规定：

$$V_H(G = 0) = 0$$

并只研究电中性的简单模型。

后期再加入严格的离子-电子总能量处理。

# 18. XC 模块

采用 LDA：

$$E_{xc} = \int n(r)\epsilon_{xc}(n(r))dr.$$

势：

$$V_{xc}(r) = \frac{\delta E_{xc}}{\delta n(r)}$$

第一版实现一个标准 LDA 参数化。

接口设计成：

```

Vxc = xc_potential(n)
Exc = xc_energy(n)

```

以后：



LDA  
↓  
PBE  
↓  
其他 functional

---

## 19. Ionic Potential

第一版不实现 PAW。

采用：

$$V_{\text{ion}}(r)$$

作为简单局域周期势。

例如：

$$V_{\text{ion}}(r) = \sum_I V_I(|r - R_I|).$$

第一阶段甚至可以采用人为构造的周期势。

这样可以把：

电子结构算法

与：

真实原子势

完全分开。

---

## 20. SCF 模块

这是整个程序的控制中心。

---

# 20.1 初始密度

可以：

## 方法 A

均匀密度：

$$n_0(r) = N_e/\Omega.$$

## 方法 B

后期根据原子密度叠加。

第一版使用 A。

---

# 21. SCF 主循环

伪代码：

```

n = initial_density()

for iteration in range(max_scf):

    VH = hartree_potential(n)
    Vxc = xc_potential(n)
    Vion = ionic_potential()

    Veff = Vion + VH + Vxc

    for kpoint in kpoints:

        eigenvalues, C = solve_eigenproblem(
            apply_H,
            ...
        )

    occupations = determine_occupations(eigenvalues)

    n_new = calculate_density(C, occupations)

    n_mixed = mix(n, n_new)

    error = convergence_error(n, n_mixed)

    if error < tolerance:
        break

n = n_mixed

```

## 22. SCF mixing

第一版：

$$n_{i+1} = (1 - \alpha)n_i + \alpha n_{\text{out}}$$

例如：

$$\alpha = 0.2.$$

后期再增加：

以及：

Kerker preconditioning

## 23. SCF 收敛判据

第一版使用：

$$\max_r |n_{\text{new}}(r) - n_{\text{old}}(r)| < \epsilon_{\text{SCF}}$$

后期增加：

- total energy convergence
- density residual
- RMS residual

## 24. Energy 模块

最终总能量不能简单地：

$$\sum_n \epsilon_n$$

因为 Kohn-Sham eigenvalues 中存在重复计数。

需要计算：

$$E_{\text{total}} = T_s + E_{\text{ion}} + E_H + E_{xc} + E_{\text{ion-ion}}$$

第一版我们会先把 electronic energy 的各个组成部分逐项实现。

这一步非常重要，因为它可以让你理解：

为什么 DOS 中的 Kohn-Sham eigenvalue 和真正的 total energy 不是一回事。

## 25. DOS 模块

第一版不急着做。

最终：

$$D(E) = \sum_{n,k} w_k \delta(E - \epsilon_{nk}).$$

数值实现使用：

$$\delta(x) \rightarrow \text{Gaussian}$$

或者其他 broadening。

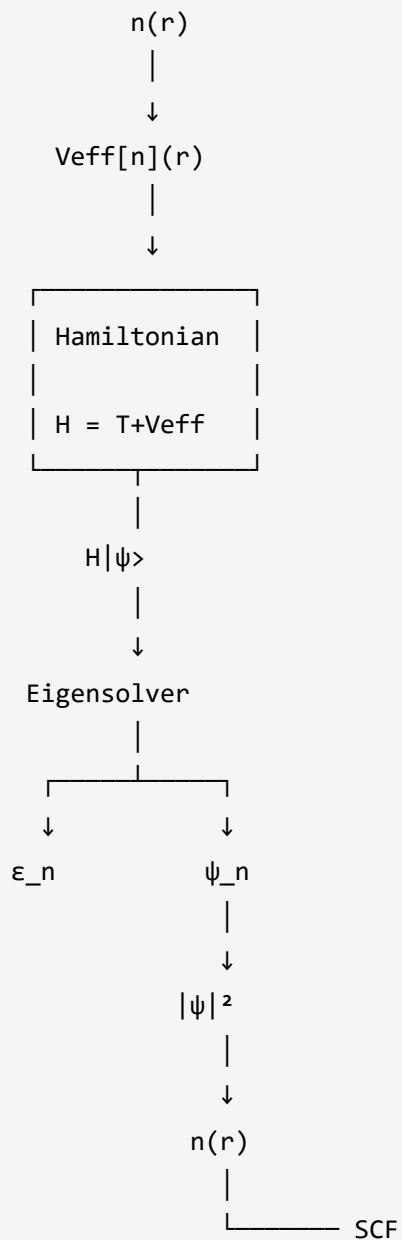
因此：

```
eigenvalues
  ↓
energy bins
  ↓
Gaussian broadening
  ↓
DOS(E)
```

---

## 26. 关键对象之间的关系

这是你以后读 VASP 源码时最应该在脑中保持的一张图：



这就是：

DFT 的自洽性

## 27. K 点处理

第一版建议：

$\Gamma$ -point only

也就是：

$$\mathbf{k} = 0.$$

这样我们首先把：

$$G$$

理解清楚。

之后再增加：

$$\mathbf{k}_1, \mathbf{k}_2, \dots$$

每个  $\mathbf{k}$  点独立解决：

$$H(\mathbf{k})C_{n\mathbf{k}} = \epsilon_{n\mathbf{k}}C_{n\mathbf{k}}.$$

最终：

$$n(r) = 2 \sum_{\mathbf{k}} w_{\mathbf{k}} \sum_n |\psi_{n\mathbf{k}}(r)|^2.$$

---

## 28. 为什么先 Gamma point?

因为如果一开始同时引入：

- (G)
- (k)
- spin
- smearing
- PAW

你会很难判断程序哪里出了问题。

Gamma-only 可以把问题简化成：

$$H(G, G')C(G) = EC(G).$$

等这个版本完全正确以后，再加入  $\mathbf{k}$  点。

---

## 29. 第一版的测试体系

建议不要直接从 Si 开始。

第一阶段测试：

## Test 1：自由电子

令：

$$V_{\text{ion}} = 0$$

并：

$$V_H = V_{xc} = 0.$$

理论结果：

$$E_G = \frac{\hbar^2 G^2}{2m}$$

你的程序应该精确恢复这个结果。

这是最重要的 sanity check。

---

## Test 2：人为周期势

例如：

$$V_{\text{ion}}(r) = V_0 \sum_i \cos(G_i \cdot r).$$

然后观察：

- band splitting
  - band gap
  - density modulation
- 

## Test 3：简单晶体模型

加入：

$$V_{\text{ion}}(r)$$

和：



$$V_H[n].$$

观察 SCF。

---

# 30. 开发阶段

整个项目分成：

## Phase 0：数学验证

实现：

$$a \rightarrow b \rightarrow G$$

---

## Phase 1：Plane-wave basis

实现：

$$E_{\text{cut}} \rightarrow G \text{ list}$$

---

## Phase 2：FFT

实现：

$$C_G \leftrightarrow \psi(r).$$

---

## Phase 3：Hamiltonian

实现：

$$C \rightarrow HC.$$

---

## Phase 4：Eigensolver

接入：

scipy eigsh.

---

## Phase 5: Density

实现：

$$\psi \rightarrow n.$$

---

## Phase 6: Hartree

实现：

$$n \rightarrow V_H.$$

---

## Phase 7: LDA

实现：

$$n \rightarrow V_{xc}.$$

---

## Phase 8: SCF

完整闭环：

$$n \rightarrow V \rightarrow H \rightarrow \psi \rightarrow n.$$

---

## Phase 9: Total Energy

验证能量收敛。

---

## Phase 10: k points

加入：

## Phase 11: DOS

从:

$$\epsilon_{nk}$$

计算:

$$D(E).$$

---

## 31. 第二阶段：开始接近真正的 DFT Code

第一版完成以后，再增加：

```
Mini-DFT
|
|— k-points
|— smearing
|— PBE
|— spin
|— pseudopotential
|— nonlocal potential
|— Davidson
|— Pulay
|— Kerker
```

---

## 32. 第三阶段：理解 VASP 的核心

最后才进入：

PAW

此时你已经有：

$$\psi$$

$$n$$

$$V_{\text{eff}}$$

$$H\psi$$

SCF

再去理解：

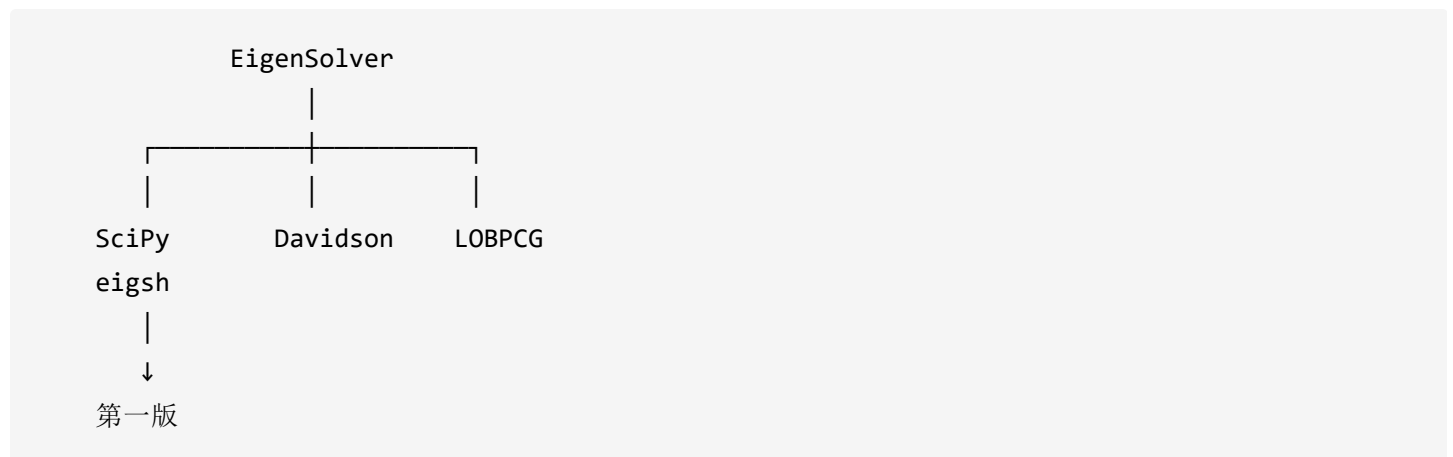
PAW transformation

就会容易很多。

否则一开始看 PAW，很容易只是在背公式。

## 33. 关于本征值求解器的最终架构

我们专门留一个接口：



第一版：

SciPy eigsh

以后自己实现：

Davidson

## 34. 最终你会理解一个非常重要的区别

整个程序里面其实有两个不同层次的迭代：

### 内层：本征值迭代

解决：

$$H[n]\psi_n = \epsilon_n\psi_n$$

例如：

- Davidson
  - Lanczos
  - LOBPCG
  - RMM-DIIS
- 

### 外层：SCF 迭代

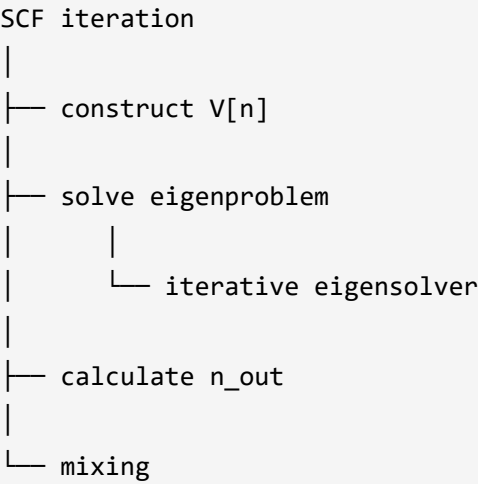
解决：

$$n_{\text{out}}[n] = n$$

例如：

- simple mixing
  - Pulay
  - DIIS
  - Kerker
- 

所以整个程序实际上是：



这个结构是你理解 VASP 的**核心中的核心**。

## 35. 第一版的最终功能列表

| 功能                      | v0.1 |
|-------------------------|------|
| 3D periodic             | ✓    |
| Plane wave              | ✓    |
| 自定义 ENCUT               | ✓    |
| Gamma point             | ✓    |
| FFT                     | ✓    |
| Matrix-free Hamiltonian | ✓    |
| SciPy eigensolver       | ✓    |
| LDA                     | ✓    |
| Hartree                 | ✓    |
| SCF                     | ✓    |
| Simple mixing           | ✓    |
| Total energy            | ✓    |
| DOS                     | 后续   |
| k points                | 后续   |

| 功能              | v0.1 |
|-----------------|------|
| Smearing        | 后续   |
| PBE             | 后续   |
| Spin            | 后续   |
| Pseudopotential | 后续   |
| Davidson        | 后续   |
| PAW             | 最后   |

## 36. 我们最终希望得到的程序逻辑

理想状态下，你运行：

```
python main.py
```

看到类似：

```
=====
Mini-DFT
=====

System:
  lattice = ...
  electrons = 2
  k-points = Gamma
  ENCUT = 30 eV

Plane-wave basis:
  N_G = 247

SCF iteration 1
  E = ...
  density error = ...

SCF iteration 2
  E = ...
  density error = ...

SCF iteration 3
  E = ...
  density error = ...

...

SCF converged.

Eigenvalues:
  band 1 = ...
  band 2 = ...

Total energy = ...
```

然后程序产生：

```
density.npy
wavefunctions.npy
eigenvalues.npy
dos.dat
```

---



# 37. 最重要的设计原则

这个项目从头到尾遵循五条原则：

## 原则 1

每个程序模块必须对应一个数学概念。

---

## 原则 2

第一版不追求真实材料，而追求可验证。

---

## 原则 3

不自己实现成熟的数值算法，先调用库。

---

## 原则 4

Hamiltonian 尽量采用 operator / matrix-free 思维。

---

## 原则 5

每完成一个模块，都必须有一个物理/数学测试。

---

# 38. 你最终应该真正理解的东西

如果这个项目完成，你应该能够从：

ENCUT

一路解释到：

$C_{n\mathbf{k}}(G)$

再到：

$\psi_{n\mathbf{k}}(\mathbf{r})$

再到：

$n(\mathbf{r})$

再到：

$V_H(\mathbf{r}), V_{xc}(\mathbf{r})$

再回到：

$H[n]$

最后形成：

$n \rightarrow H[n] \rightarrow \psi \rightarrow n$

的自治闭环。

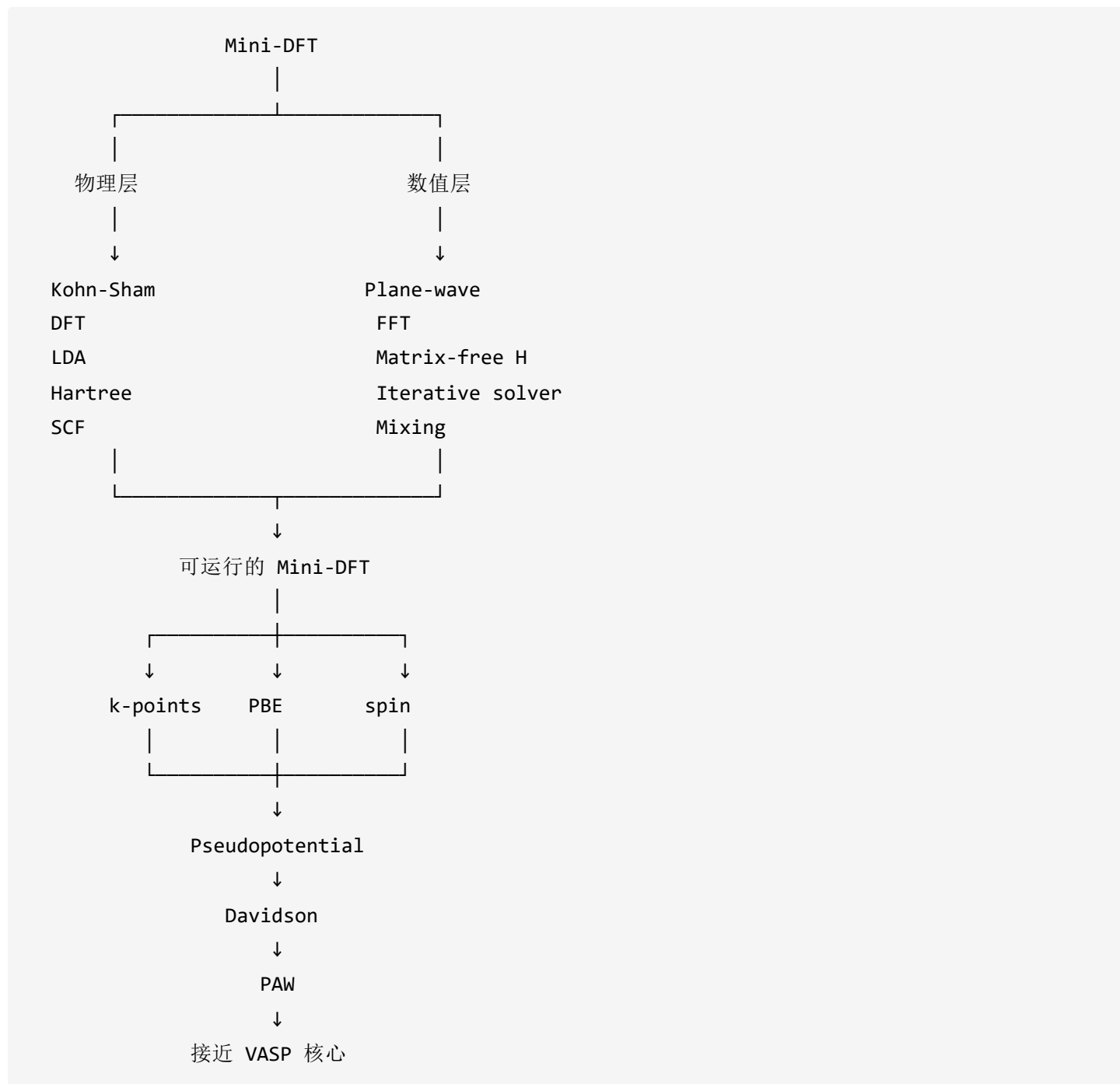
而在这个闭环内部，你也会真正理解：

$Hc \rightarrow \text{iterative eigensolver}$

为什么可以不显式构造巨大矩阵，以及为什么 VASP 可以在极大的平面波空间里工作。

---

# 最终路线图



**下一步就应该开始 Phase 0:** `lattice.py + basis.py`。我们先不写 SCF，甚至不写 DFT；只用几十行代码把“给定晶格 + 给定 ( $E_{\text{cut}}$ )  $\rightarrow$  自动生成所有 (G)  $\rightarrow$  得到 ( $N_{\text{PW}}$ )”完整实现，并同时检查它和数学公式是否一一对应。这样后面每一步都是在这个地基上往上盖。